HLD Level 1.3

Load Balancing & Service Discovery - C++ / Systems Interview Notes

> **Goal: understand how traffic is distributed across multiple service instances, how unhealthy instances are removed, and how dynamically changing services find each other.**

1. Why load balancing exists

A single server has finite CPU, memory, network bandwidth, file descriptors and failure tolerance. To scale and provide redundancy, we commonly run multiple instances and put a load-balancing layer in front.

```
Client -> Load Balancer -> Server A
-> Server B
-> Server C
```

The LB provides a stable entry point and chooses an eligible backend. It improves scalability and availability, but does not remove bottlenecks in shared databases, queues or storage.

2. Vertical vs horizontal scaling

Vertical: give one machine more CPU/RAM/resources.
Horizontal: add more service instances. C++ efficiency helps, but one extremely fast process remains one failure domain.

3. What are we balancing?

> **Always ask: are we balancing HTTP requests, TCP connections, UDP flows, or application jobs? The balancing unit affects the algorithm and failure behavior.**

4. L4 vs L7 load balancing

L4: makes routing decisions using transport/network information such as TCP/UDP endpoints. Useful when application payload inspection is unnecessary.

L7: understands an application protocol such as HTTP and can route by host, path, headers or other application information.

```
/images/* -> Image Service
/orders/* -> Order Service
/users/* -> User Service
```

A custom C++ TCP gateway may naturally use transport-level balancing, while HTTP services often benefit from application-aware L7 routing.

5. Common balancing algorithms

Round robin: rotate across servers. Good starting point for similar servers and similar request costs.

Weighted round robin: send proportionally more traffic to stronger servers.

Least connections: prefer the backend with fewer active connections; useful for long-lived/uneven connections.

Hash-based: hash a client/device/key to select a backend; can provide affinity but membership changes require care.

> **There is no universally best algorithm. Tie your choice to workload characteristics.**

6. Why round robin can be insufficient

Requests can have very different costs. One backend may be processing a 10-second operation while another finishes requests in milliseconds. Long-lived connections can also make simple request rotation misleading.

7. Health checks: liveness vs readiness

Liveness: is the process alive?
Readiness: is it currently capable of serving traffic?

```
LB -> A : READY
LB -> B : NOT READY
LB -> C : READY

New traffic -> A or C
```

A process can be alive but not ready because it is starting, overloaded, warming up, or cannot reach a critical dependency.

8. Active and passive health

Active: periodically probe an endpoint/connection.
Passive: observe real request failures, timeouts and connection errors. Systems may combine both. Avoid overly aggressive checks that cause unhealthy/healthy flapping.

9. Stateless services

Scaling is simplest when any healthy instance can handle any request. Durable/shared state is kept in an external database, cache or storage system.

```
Client -> LB -> A / B / C
|
Shared DB/Cache
```

If session state exists only in Server A memory, a later request routed to B may fail. This motivates stateless design or explicit affinity/state strategies.

10. Sticky sessions

Session affinity attempts to route a client repeatedly to the same backend. It can simplify some stateful workloads, but can cause uneven load and harder failover; local state may disappear if that backend dies.

11. Long-lived C++ connections

For device gateways, WebSockets and custom TCP systems, the LB may distribute **connections**, not individual messages. Once established, traffic normally stays with that backend for the connection lifetime.

```
100k devices -> LB -> Gateway A: 34k
-> Gateway B: 33k
-> Gateway C: 33k
```

Adding Gateway D does not automatically redistribute all existing persistent TCP connections. New connections can favor D; existing connections may remain until reconnect or drain.

12. Connection draining

Before maintenance/deployment: mark an instance not ready for new traffic, stop assigning new connections, let existing work/connections finish where possible, then close/reconnect remaining clients according to policy and shut down.

Abruptly killing a gateway can disconnect thousands of clients and trigger a reconnect storm.

13. Load balancer availability

A single irreplaceable LB becomes a single point of failure. Production architectures use redundancy/failover for the load-balancing layer itself.

```
Clients -> LB layer (redundant) -> Service instances
```

14. Service discovery

Instance addresses change as services start, stop, scale and fail. Service discovery answers: **which instances currently provide this service, and where are they?**

```
Service A -> Discovery/Registry -> B1
-> B2
-> B3
```

15. Registration/discovery lifecycle

Typical flow: instance starts -> becomes registered/discoverable -> health is maintained -> callers resolve healthy instances -> failed/terminated instances stop being eligible.

16. Client-side vs server-side discovery

Client-side: caller obtains instances and chooses one itself.
Server-side: caller sends to a proxy/LB that performs discovery and selection.

```
Client-side: A -> Registry -> [B1,B2,B3] -> B2

Server-side: A -> LB/Proxy -> B1/B2/B3
```

Server-side discovery keeps routing logic out of application code. Client-side discovery gives direct control but increases client complexity.

17. DNS-based discovery

DNS can map a service name to one or more addresses. Caching and TTLs mean membership changes may not become visible immediately.

18. Hotspots and uneven load

Traffic can remain uneven because clients differ in activity, requests differ in cost, sticky sessions concentrate users, or hash keys become hot. Ask whether your balancing metric actually represents work.

19. Overload and backpressure

An LB cannot manufacture capacity. If every backend is saturated, consider admission control, rate limiting, bounded queues, load shedding, autoscaling, prioritization or fast overload responses rather than unlimited queue growth.

20. C++ systems deep dive

For a high-performance gateway, expect questions about persistent socket counts, file-descriptor limits, per-connection buffer memory, epoll/event loops, worker pools, connection draining, reconnect storms, bounded work queues and distribution after scaling.

```
50,000 connections x 16 KB app buffer ~= 800 MB
(before kernel buffers, metadata and other process memory)
```

21. Reconnect storm

If a gateway holding 30,000 device connections fails, thousands of clients may reconnect simultaneously, causing spikes in connection setup, authentication, TLS, CPU and downstream traffic.

```
Gateway failure -> 30k disconnects -> simultaneous reconnect -> overload
```

Mitigations include exponential/randomized backoff, spare capacity, connection admission control and staged recovery.

22. Interview decision checklist

1. What are we balancing: requests, connections or jobs?
2. Are servers equal capacity?
3. Are requests equal cost?
4. Are connections long-lived?
5. Is affinity required?
6. How is health/readiness determined?
7. What happens when an instance fails?
8. How are instances discovered?
9. How are deployments drained?
10. What if every backend is overloaded?

23. Common mistakes

LB means infinite scale: false.
Round robin always works: false.
Alive means ready: false.
Sticky sessions solve state: they add trade-offs.
New server rebalances existing TCP connections: not automatically.
Ignore LB failure: wrong; the LB layer needs redundancy.
Ignore reconnect storms: dangerous for device fleets.

24. Practice scenarios

1. Three identical stateless HTTP servers: what simple balancing algorithm would you start with and why?

2. Three gateways have 40k, 10k and 5k long-lived TCP connections. Is round robin ideal for new connections?

3. A process is alive but cannot reach its required database. Should it receive traffic? Explain liveness vs readiness.

4. Add a fourth gateway to 90k existing persistent connections. Do all existing connections automatically rebalance?

5. One gateway fails and 30k devices reconnect simultaneously. What risks and mitigations do you see?

6. Why is a stateless service generally easier to horizontally scale than a stateful one?

25. 60-second revision

LB: distributes traffic across eligible backends.
L4: transport-level routing.
L7: application-aware routing.
Round robin: simple rotation.
Least connections: useful for uneven/long-lived connections.
Liveness: process alive.
Readiness: able to serve traffic.
Sticky session: affinity with trade-offs.
Service discovery: locate current instances.
Draining: stop new traffic before shutdown.
C++ focus: sockets, buffers, FDs, event loops and reconnect storms.

26. Ready for Level 1.4 when...

You can explain why an LB is needed, select a balancing strategy from workload characteristics, distinguish L4/L7, reason about readiness/health, explain discovery, and handle persistent-connection

draining/failure scenarios.